

Contents

1. ABSTRACT	2
2. INTRODUCTION	2
2.1 What is Stack Overflow?	2
2.2 What are tags?	2
2.3 Why is tag prediction important?	2
3. OBJECTIVE	3
4. CONCEPTS	3
4.1 Natural Language Processing	3
4.2 Multi-Label Classification	4
4.3 TF-IDF	4
5. DATA	5
5.1 Data Loading	5
5.2 Data Pre-processing	6
6. EXPLORATORY DATA ANALYSIS	7
6.1 Data Exploration	7
6.2 Topic Modelling	10
6.2.1 C#	10
6.2.2 Java	11
6.2.3 PHP	11
6.2.4 Java Script	12
6.2.5 Android	13
7. MODEL BUILDING	14
7.1 Multilabel Classification using OneVsRest Classifier	14
7.2.1 SGD Classifier	14
7.2.2 LogisticRegression Classifier	14
8. EVALUATION METRICS	15
8.1 Accuracy	15
8.2 Micro-Averaged F1 Score	15
8.3 Macro-Averaged F1-score	16
8.4 Hamming Loss	16
9. RESULTS AND DISCUSSION	17
10. CONCLUSION	18
11. CODES	19
12. BIBLIOGRAPHY	24

NLP BASED TAG PREDICTION

1. ABSTRACT

One of the most used and highly trusted websites by every developer/engineer/student is Stack Overflow for learning and sharing their knowledge in computer programming. Through this project, we have aimed at creating a model for automatically predicting the tags associated with the posts published on the site to categorize them for easy performance and better user experience. We also learned about the problem of multi-label classification and used the models suitable for it to achieve the aim of the project.

2. INTRODUCTION

2.1 What is Stack Overflow?

Stack Overflow is a question-and-answer website used by



professionals and enthusiastic programmers. It is one of the most actively viewed sites in the Stack Exchange, which is a network of question-and-answer websites on several topics in diverse fields, each site covering a specific topic where questions, answers, and users are subject to a reputation award process. Stack Overflow features questions and answers on a wide range of topics under computer programming. The website serves as a platform for users to ask and answer questions, and through membership and active participation they can earn reputation points and badges. By increasing these points and badges, users can unlock new privileges like the ability to vote questions and answers up or down, and edit and comment on other people's posts.

2.2 What are tags?

Tags are simple keywords or labels that are assigned to a piece of information that helps to describe an item and allows it to be found again while browsing or searching. The tags in stack overflow help to categorize a question with other similar questions and describe the topic of the question.

2.3 Why is tag prediction important?

These tags are extremely important for the proper working of Stack Overflow. They are a means of connecting experts with questions that they will be able to answer by sorting questions into specific, well-defined categories or topics. They are also used to identify all

those questions and answers that are relevant to people's search queries. Clicking on a tag under a question can also take the user to a page showing all the questions within that tag.

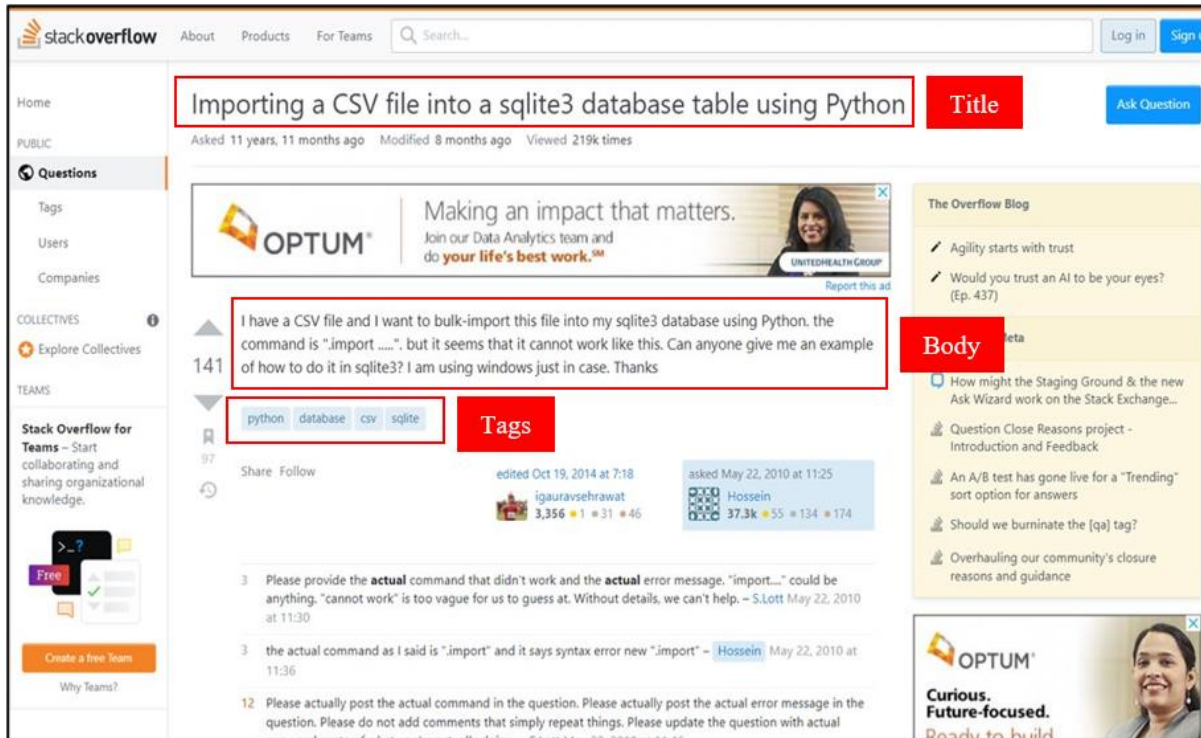


Figure 1: User Interface of Stack Overflow website, highlighting the title, body, and tags of a question.

3. OBJECTIVE

The main objective of this project is to predict tags for each of the given questions on the Stack Overflow website using its title and body. Also, to attain the same objective by making use of Natural language processing techniques and identify what kind of problem it is and perform the most suitable models for predicting with maximum accuracy.

4. CONCEPTS

4.1 Natural Language Processing

Natural Language Processing (NLP) is the branch of computer science specifically Artificial Intelligence (AI) that is concerned with providing computers the ability to understand the text and spoken words in the same way as human beings can. It helps computers communicate with humans in their language and scales other language-related tasks such as

reading text, hearing speech, interpreting it, measuring sentiment, and determining which parts are important.

Today's machines can analyse language-based data without fatigue and in a consistent unbiased way. Considering the staggering amount of unstructured data that is generated every day, automation will be critical to comprehensively analysing text and speech data efficiently. While supervised, unsupervised learning and deep learning are now widely used for modelling human language, there's also a need for syntactic and semantic understanding and domain expertise which are not necessarily present in these machine learning approaches. NLP is important because it helps to resolve the ambiguity in language and adds useful numeric structure to the data for many downstream applications like speech recognition and text analytics. NLP tasks break down language into shorter elemental pieces, try to understand the relationships between these pieces, and explore how they work together to create meaning. The overarching goal of NLP tasks is to take raw language input and use linguistics and algorithms to transform or enrich the text in such a way that it delivers greater value.

4.2 Multi-Label Classification

Classification is a predictive modelling problem that involves outputting a class label given some input. It is different from the regression problem where a numeric value is predicted rather than class labels. A classification problem might involve predicting single, two, or more class labels which are called normal, binary, and multiple-classification respectively. In these classifications, the classes are mutually exclusive. The classification problem which predicts more than two class labels is called multi-class classification. Those multiple classification problems where the class labels are not mutually exclusive i.e., zero or more labels are required as output for each input sample, and the outputs are required simultaneously, are called multi-label classification. It is assumed that the outputs are a function of the inputs.

4.3 TF-IDF

Term Frequency-Inverse Document Frequency is a text vectorizer that combines 2 concepts – Term Frequency (TF) and Document Frequency (DF). TF is the number of occurrences of a specific term in a document and indicates how important that term is in the document. It represents every text from the data as a matrix whose rows are the numbers of documents and columns are the number of distinct terms throughout all documents. DF is the number of documents containing a specific term and indicates how common the term is. Inverse

document frequency(IDF) is the weight of a term, and aims to reduce the weight of a term if its occurrences are scattered throughout all the documents.

5. DATA

The data we have used for this project has been collected from the Keyword Extraction competition by Facebook in Kaggle:

<https://www.kaggle.com/competitions/facebook-recruiting-iii-keyword-extraction/data>

The train data contains 4 columns: Id, Title, Body, and Tags.

- Id – Unique identifier for each question
- Title – Title of each question
- Body – Body or description of each question
- Tags – Tags associated with each question

The questions in the data are randomized and also no filtering has been performed on the data.

5.1 Data Loading

The data originally contains 6034195 rows, hence the size of 6.5 GB. Having a huge file size and lesser RAM size posed trouble in loading data into python. Hence, we decided to take only a random 50% of the original data.

Since there is a large amount of data, loading the data directly from the csv file using pandas is time-consuming. SQLite is a self-contained, fast, and reliable file-based SQL database. It comes bundled with Python and can be used by any Python applicant without having to install any additional software. By using the sqlite3 library in python, we created a connection to an SQLite database and loaded the randomly selected data into it. After that, the data was read as a pandas data frame for performing further processes.

	Id	Title	Body	Tags
0	1	How to check if an uploaded file is an image w...	<p>I'd like to check if an uploaded file is an...	php image-processing file-upload upload mime-t...
1	3	R Error Invalid type (list) for variable	<p>I am import matlab file and construct a dat...	r matlab machine-learning
2	4	How do I replace special characters in a URL?	<p>This is probably very simple, but I simply ...	c# url encoding
3	5	How to modify whois contact details?	<pre><code>function modify(.....)\n{\n \$mco...	php api file-get-contents
4	6	setting proxy in active directory environment	<p>I am using a machine on which active direct...	proxy active-directory jmeter
5	8	How to fetch an XML feed using asp.net	<p>I've decided to convert a Windows Phone 7 a...	c# asp.net windows-phone-7

Figure 2: Snippet of the data

5.2 Data Pre-processing

The data contains much redundant information which needs to be removed for the better performance of our models. To begin with, we removed the unwanted column in our data – column Id. Secondly, we checked if any null values were present. There were only 6 null values, hence them proving to be insignificant. Thus, we dropped them out of the table. Thirdly we checked for duplicate values. There were 4,91,854 duplicate rows present and we dropped them all too. Since our data is that of questions regarding technical and non-technical computer programming questions, they had so many characters that are irrelevant for natural language processing. Hence, we removed all such characters – hyperlinks, code segments, HTML segments, and punctuations. We also removed all the stop words. Finally, we converted all the remaining cleaned data to lowercase. After the process of data cleaning, the data had the dimension of 25,25,244 x 4 rows and columns.

	Title	Body	Tags
0	check uploaded file image without mime type	check uploaded file image file png jpg jpeg gi...	php image-processing file-upload upload mime-t...
1	error invalid type list variable	import matlab file construct data frame matlab...	r matlab machine-learning
2	replace special characters url	probably simple simply cannot find answer basi...	c# url encoding
3	modify whois contact details	using modify function displays warning message ...	php api file-get-contents
4	setting proxy active directory environment	using machine active directory configured deve...	proxy active-directory jmeter
5	fetch xml feed using asp net	decided convert windows phone app fetches xml ...	c# asp.net windows-phone-7
6	sql server procedure call inline concatenation...	using sql server wondering another way writing...	sql variables parameters procedure calls

Figure 3: Sample of the cleaned data

For tag prediction, we have to consider both the title as well as the body of the questions. Hence, we combine the columns – Title and Body into a column named Text by concatenating using the space as a separator.

6. EXPLORATORY DATA ANALYSIS

6.1 Data Exploration

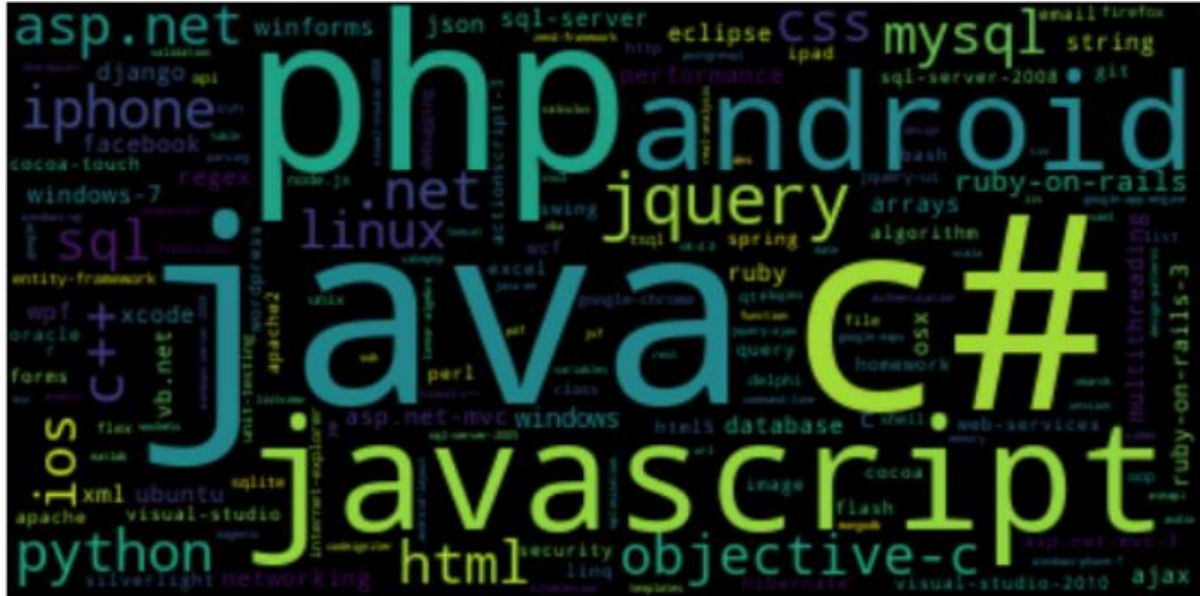


Figure 4: Word cloud of the text in the title and body of the questions

The word cloud shows that c#, java, php, JavaScript, and android are some of the most frequently occurring tags.

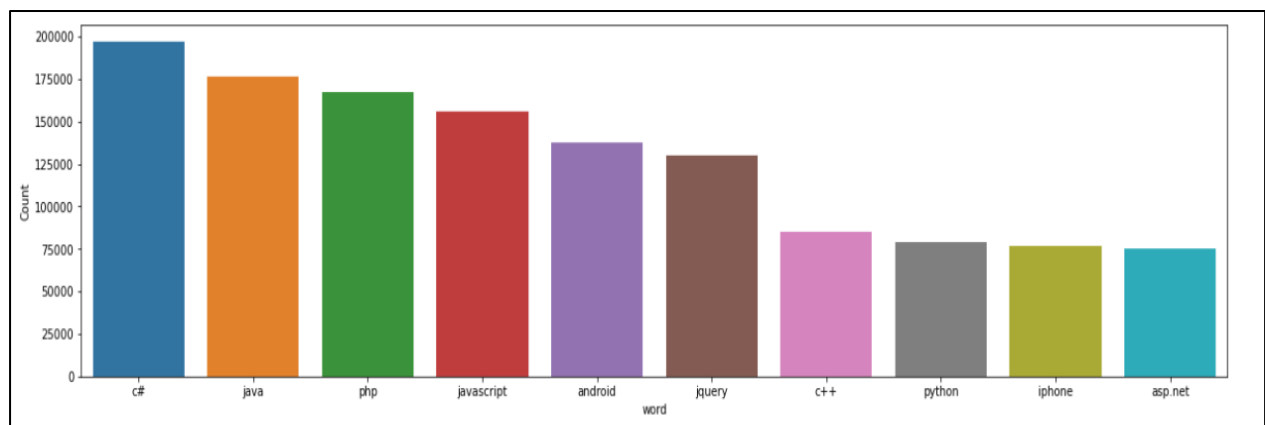


Figure 5: Bar Chart of Top 10 Frequently Occurring Tags

Most of the Frequently occurring tags are of Programming languages. C# is the top most frequently occurring programming language and Android is the most frequently occurring tag in Operating systems.

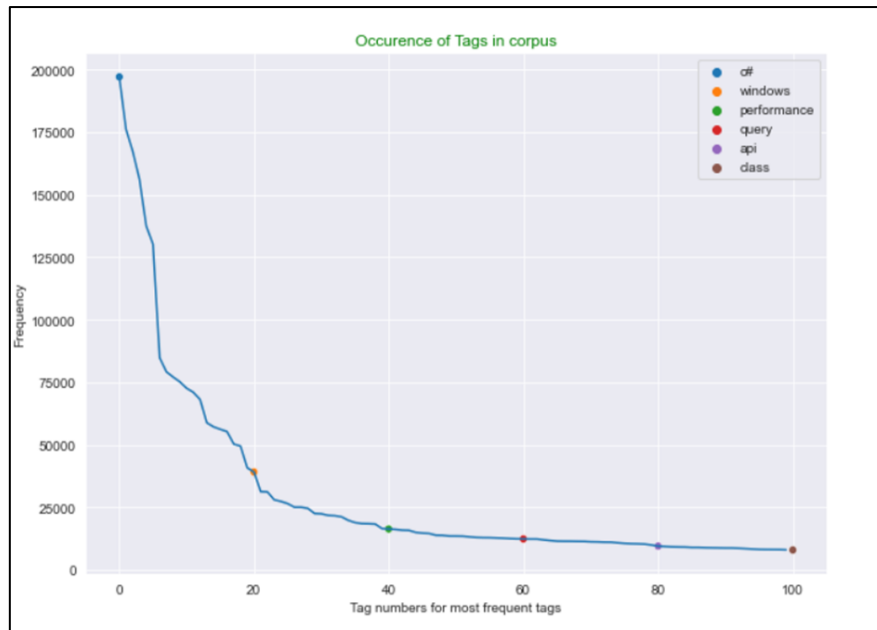


Figure 6: Occurrence of Top 100 Frequent Tags

Most of the Tags have occurred 200000 times. After that, the frequency of the tags started decreasing. C# is the most frequently occurring tag. The frequency of the tags starts decreasing from 20th Tag.

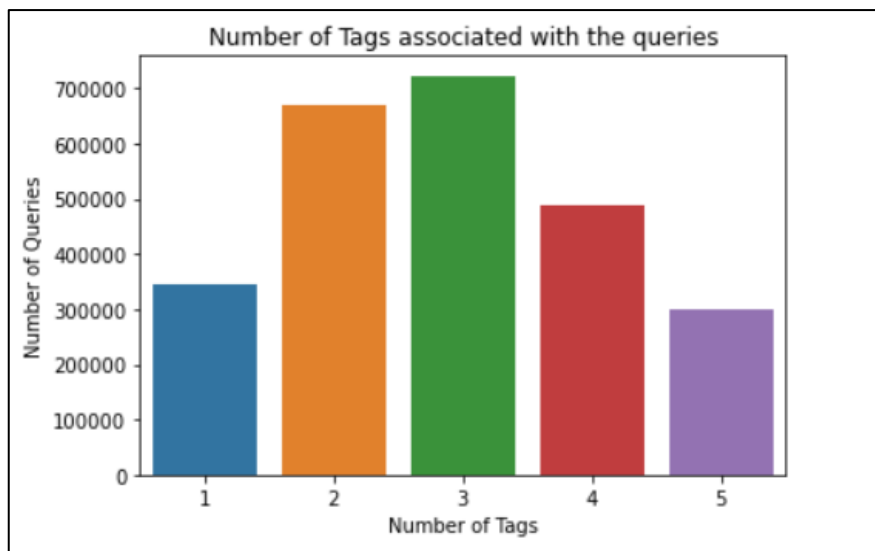


Figure 7: Distribution of Number of Tags associated with the Queries

The maximum number of tags per question is 5 and minimum number of tags per question is 1. Most of the questions are having 2 or 3 tags.

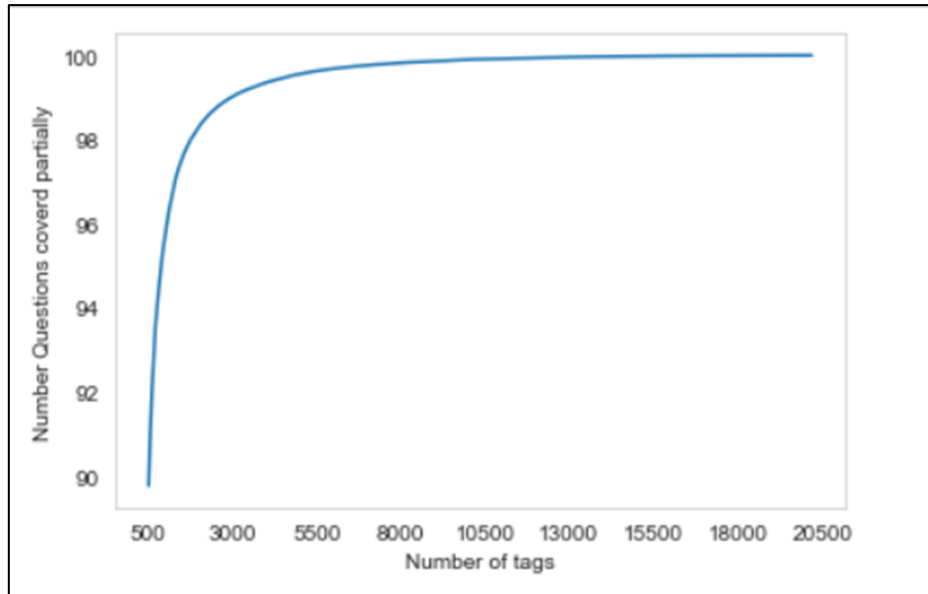


Figure 6: Number of Questions covered by the Number of Tags

#Tags	%Ques
500	94.023
1000	95.769
1500	96.872
2000	97.554
2500	97.998
3000	98.308
3500	98.554
4000	98.741
4500	98.894
5000	99.02
5500	99.129
6000	99.21
6500	99.282
7000	99.356

Number of tags in sample: 39975

Number of tags taken: 5500 (13.758599124452783 %)

Figure 7: Tabular form of Tags and the percentage of questions covered

There are 39975 tags in the data but we have taken only 5500 tags. With 5500 tags we are covering 99% of the questions and after that, all other tags are covering approximately the same percentage of questions. By choosing 5500 tags out of 39975 tags we are only losing 1% of the questions.

6.2 Topic Modelling

Topic modelling is an unsupervised machine learning technique that is capable of scanning a set of documents, detecting word and phrase patterns within them, and automatically clustering word groups and similar expressions that best characterize a set of documents. This is known as “Unsupervised” Machine Learning because it doesn’t require a pre-existing list of tags or training data that has been previously categorized by humans.

This type of modelling is very much useful when there are many documents present and when we want to know what type of information is present in them. This takes a lot of time when done manually and this can be done easily in very little time using Topic Modelling.

Latent Dirichlet Allocation (LDA) is a popular tool for Topic Modelling and it’s used to classify text in a document to particular topics. The purpose of LDA is to map each document in a corpus to a set of topics that covers a good deal of the words in a document.

We performed topic modelling on our data to see if it provides any useful insights for our goal. The following are the results of topic modelling and the meaningful sentences that the topics explain:

6.2.1 C#

```
[(0,
  '0.020*"application" + 0.018*"c" + 0.015*"file" + 0.013*"service" + '
  '0.013*"using" + 0.013*"net" + 0.011*"web" + 0.011*"code" + 0.011*"windows" '
  '+ 0.011*"server"'),
 (1,
  '0.018*"control" + 0.017*"form" + 0.014*"event" + 0.013*"button" + '
  '0.013*"code" + 0.012*"image" + 0.012*"want" + 0.011*"user" + 0.009*"click" '
  '+ 0.009*"text"'),
 (2,
  '0.026*"data" + 0.018*"database" + 0.018*"file" + 0.016*"using" + '
  '0.016*"page" + 0.013*"sql" + 0.013*"table" + 0.012*"code" + 0.012*"asp" + '
  '0.012*"net"'),
 (3,
  '0.022*"c" + 0.012*"code" + 0.011*"way" + 0.008*"use" + 0.008*"string" + '
  '0.008*"one" + 0.008*"using" + 0.006*"time" + 0.006*"want" + '
  '0.005*"example"'),
 (4,
  '0.030*"class" + 0.023*"object" + 0.021*"method" + 0.018*"type" + '
  '0.012*"code" + 0.010*"value" + 0.010*"list" + 0.010*"use" + 0.010*"using" + '
  '0.009*"property"')]
```

- Code file net application using web service on windows service
- Code of control for the button click event
- Asp.net using database as SQL table

- Example c code using time as a string
- Code for class object method type as string

6.2.2 Java

```
[ (0,
  '0.025*"javascript" + 0.023*"page" + 0.016*"jquery" + 0.016*"code" + '
  '0.014*"html" + 0.012*"using" + 0.010*"want" + 0.010*"function" + '
  '0.010*"button" + 0.009*"click"'),
  (1,
    '0.023*"server" + 0.014*"using" + 0.013*"application" + 0.013*"client" + '
    '0.012*"data" + 0.011*"web" + 0.011*"user" + 0.009*"request" + 0.008*"app" + '
    '0.008*"thread"'),
  (2,
    '0.011*"java" + 0.009*"code" + 0.009*"use" + 0.008*"way" + 0.007*"using" + '
    '0.007*"one" + 0.006*"time" + 0.005*"want" + 0.005*"make" + '
    '0.005*"question"'),
  (3,
    '0.017*"class" + 0.016*"object" + 0.014*"method" + 0.013*"code" + '
    '0.012*"string" + 0.011*"value" + 0.011*"array" + 0.011*"java" + '
    '0.010*"using" + 0.010*"want"'),
  (4,
    '0.050*"java" + 0.041*"file" + 0.014*"error" + 0.014*"files" + 0.012*"using" '
    '+ 0.012*"xml" + 0.011*"project" + 0.011*"run" + 0.010*"application" + '
    '0.010*"eclipse"') ]
```

- Using Javascript, JQuery, html clicks the want function button
- App thread using client data application and web user request
- One time use Java Code and want to make a question
- Object class, method code, and value array are using Java
- Java error files using XML project to run application Eclipse

6.2.3 PHP

```
[
(0,
'0.030*"php" + 0.014*"function" + 0.014*"code" + 0.013*"string" + '
'0.012*"use" + 0.011*"class" + 0.009*"way" + 0.008*"using" + 0.007*"error" + '
'0.006*"object"'),
(1,
'0.069*"array" + 0.034*"file" + 0.031*"image" + 0.031*"php" + 0.019*"xml" + '
'0.015*"using" + 0.015*"images" + 0.015*"want" + 0.014*"code" + '
'0.012*"upload"'),
(2,
'0.024*"database" + 0.024*"table" + 0.023*"mysql" + 0.018*"query" + '
'0.015*"data" + 0.012*"user" + 0.011*"want" + 0.010*"one" + 0.009*"field" + '
'0.009*"php"'),
(3,
'0.050*"php" + 0.023*"server" + 0.019*"file" + 0.016*"error" + 0.011*"using" '
'+ 0.011*"script" + 0.011*"files" + 0.007*"web" + 0.006*"run" + '
'0.006*"application"'),
(4,
'0.032*"page" + 0.030*"php" + 0.016*"form" + 0.016*"code" + 0.016*"user" + '
'0.012*"url" + 0.012*"using" + 0.010*"post" + 0.010*"want" + 0.009*"html"')]
]
```

- Code string for PHP function using class way and an error object
- Array of images in php xml using images want to upload code
- Table in database mysql query data user want one field in PHP
- Error occurred while running web application using script files on the
- Html, php is using url post code in a user form.

6.2.4 Java Script

```
[
(0,
'0.033*"data" + 0.024*"php" + 0.021*"ajax" + 0.019*"using" + 0.018*"form" + '
'0.015*"json" + 0.014*"file" + 0.012*"server" + 0.012*"javascript" + '
'0.012*"table"'),
(1,
'0.040*"page" + 0.017*"javascript" + 0.016*"code" + 0.013*"button" + '
'0.011*"window" + 0.011*"using" + 0.011*"user" + 0.010*"script" + '
'0.009*"html" + 0.009*"load"'),
(2,
'0.020*"div" + 0.019*"jquery" + 0.018*"image" + 0.016*"text" + 0.013*"want" '
'+ 0.011*"element" + 0.011*"using" + 0.011*"html" + 0.010*"click" + '
'0.009*"code"'),
(3,
'0.035*"function" + 0.026*"javascript" + 0.022*"code" + 0.019*"object" + '
'0.014*"array" + 0.010*"jquery" + 0.010*"variable" + 0.010*"string" + '
'0.009*"value" + 0.009*"error"'),
(4,
'0.028*"javascript" + 0.010*"use" + 0.010*"way" + 0.009*"using" + '
'0.007*"web" + 0.006*"one" + 0.006*"time" + 0.005*"file" + 0.005*"files" + '
'0.005*"code"')]
]
```

- Javascript data table with php ajax using json file server
- Using user javascript to load the html code for the button window in the page
- Using html click code, div jquery image text want an element
- String value error in javascript function object array jquery variable

- Use Javascript with one-time code files on the web

6.2.5 Android

```
[(0,
  '0.019*"android" + 0.015*"data" + 0.014*"code" + 0.013*"database" + '
  '0.012*"using" + 0.011*"image" + 0.010*"want" + 0.009*"use" + 0.009*"string" '
  '+ 0.008*"way"'),
(1,
  '0.017*"view" + 0.016*"android" + 0.015*"layout" + 0.015*"button" + '
  '0.013*"screen" + 0.013*"want" + 0.012*"xml" + 0.011*"code" + '
  '0.011*"listview" + 0.011*"list"'),
(2,
  '0.052*"android" + 0.020*"file" + 0.017*"app" + 0.012*"application" + '
  '0.012*"using" + 0.011*"project" + 0.010*"code" + 0.009*"device" + '
  '0.008*"eclipse" + 0.008*"error"'),
(3,
  '0.022*"activity" + 0.021*"android" + 0.021*"app" + 0.016*"application" + '
  '0.012*"user" + 0.010*"service" + 0.010*"code" + 0.010*"server" + '
  '0.009*"want" + 0.009*"using"'),
(4,
  '0.081*"java" + 0.050*"android" + 0.040*"error" + 0.022*"androidruntime" + '
  '0.022*"exception" + 0.019*"video" + 0.017*"method" + 0.016*"class" + '
  '0.013*"app" + 0.013*"code"')]
```

- Data code database for Android using image want to use string
- Listview list with android layout button screen want XML code
- The Android file app application is using project code device eclipse error
- Activity Android using application code server want to use server.
- An android runtime exception has occurred in the video method of the class app code in Java

7. MODEL BUILDING

Most of the machine learning algorithms are fulfilled with mathematical concepts. Hence, they expect the data to be numerical such as 2-dimensional array with rows as instances and columns as features. The natural language processing involves data that is in the form of raw text which none of the machine learning algorithms and computers can understand. The process of transforming text into a vector is called as text vectorization. We used the text vectorization algorithm called TFIDF-vectorizer which is a popular and easy approach for transforming text into vectors and to further perform the different models.

7.1 Multilabel Classification using OneVsRest Classifier

As we know, this is a multilabel classification problem and each document may have one or more defined tags simultaneously. We already saw that several datapoints mostly have 2 or 3 tags.

Most traditional ML algorithms are developed for single-label classification problems. There are a lot of approaches in the literature to transform the multi-label problem into multi single-label problems so that the existing single-label algorithms can be used. One such technique is the One-Vs-Rest (OvR) multi-label classifier, also known as One-Vs-All. In OneVsRest classifier, we fit one classifier per class and for each classifier, the class is fitted against all other classes. Since each class is represented by one and one classifier only, it is possible to gain knowledge about the class by inspecting its corresponding classifier. Hence computational efficiency and interpretability are the two advantages of this approach.

The scikit-learn library in python provides a separate OneVsRestClassifier class that allows the one-vs-rest strategy to be used with any other classifier.

7.2.1 SGD Classifier

Stochastic Gradient Descent (SGD) is a simple and very efficient approach to fitting the linear classifiers and regressors under convex loss functions. SGD is merely an optimization technique and does not correspond to a specific family of ML models. The function SGDClassifier in python can implement a plain stochastic gradient descent learning routine which supports different loss functions and penalties for classification.

7.2.2 LogisticRegression Classifier

As we saw in the EDA, 500 tags cover almost 94% of the question and 5500 tags cover 99%.

8. EVALUATION METRICS

In this section, we will discuss the several evaluation metrics that we consider for assessing and comparing the performance of our models.

8.1 Accuracy

Accuracy in simple terms refers to how often something achieves some goal. When it comes to classification, accuracy is measured by how often a model correctly classifies data. So for a classification problem accuracy can be measured as the number of correct predictions divided by the total number of predictions. More precisely, it is the sum of the number of true positives (TP) and true negatives (TN), divided by the number of examples in the dataset.

$$\text{Accuracy} = \frac{TP+TN}{FP+FN+TP+TN}$$

Accuracy can be a misleading metric for imbalanced datasets.

8.2 Micro-Averaged F1 Score

Micro F1-score (short for Micro-averaged F1 score) is used to assess the quality of multilabel classification problems. It measures the F1 score of the aggregated contributions of all the classes.

It is performed by first calculating the sum of all true positives (TP), false positives (FP), true negatives (TN) and false negatives (FN) over all the labels. Then we compute the micro-precision and micro-recall from the sums. And finally, we calculate the harmonic mean to get the micro F1-score.

$$\text{Micro F1-score} = 2 * \frac{\text{Micro-precision} * \text{Micro-recall}}{\text{Micro-precision} + \text{Micro-recall}}$$

Micro-averaging will put more emphasis on the common labels in the dataset since it gives each sample the same importance which is a preferred behaviour for multi-label classification problems. Labels that are very rare may not be intended to influence the overall F1 score heavily if the model is performing well on the other more common labels.

Micro F1-score = 1 is the best value (i.e., perfect micro-precision and micro-recall) and the worst value is 0. Micro-averaged values can be high even if the model is performing very poorly on a rare label since it gives more weight to the common labels.

8.3 Macro-Averaged F1-score

Macro F1-score (short for macro-averaged F1 score) is also used to access the quality of multilabel classification problems. It is performed by first computing the F1 score per class/label and then averaging them.

$$\text{Macro F1-score} = \frac{1}{N} \sum_{i=0}^N F1 - score_i$$

Macro-averaging will give same importance to each class/label. Here, all classes contribute equally regardless of how often they appear in the dataset.

Macro F1-score = 1 is the best value and the worst value is 0. It will be low for models that only perform well on the common classes while performing poorly on the rare classes.

8.4 Hamming Loss

In multi-label classification, a misclassification is no longer a hard wrong or right. A prediction containing a subset of the actual classes should be considered better than a prediction that contains none of them. Hamming loss is the fraction of wrong labels to the total number of labels. It reports how many times on average, the relevance of an instance to a class label is incorrectly predicted. It takes into account the prediction error (an incorrect label is predicted) and missing error (a relevant label not predicted), normalized over total number of classes and total number of instances.

Hamming loss penalizes only the individual labels in a multi-label output. Hence hamming loss is a good measure of model performance. Lower the hamming loss, better is the model performance.

9. RESULTS AND DISCUSSION

We ran the different models for the data in many different ways and also found the different evaluation metrics for all those. The following are the results we got:

Model	Accuracy	Micro F1 score	Macro F1 score	Hamming loss
OvR-SGD, labels=500	0.2170	0.4209	0.2973	0.0029
OvR- SGD, labels=5000	0.0818	0.3598	0.1134	0.0004
OvR-SGD, labels=500, 3*title	0.2174	0.4209	0.2962	0.0029
OvR-SGD, labels=5000, 3*title	0.8179	0.3901	0.1521	0.0004
OvR- LogisticRegress ion, labels=500	0.2268	0.4468	0.3277	0.0028
OvR- LogisticRegress ion, labels=500, 3*title	0.2271	0.4471	0.3272	0.0028
OvR- LogisticRegress ion, labels=5000, 3*title	0.0879	0.3901	0.1521	0.0004

10. CONCLUSION

As part of our project, we collected the data of questions and their title and descriptions published in the Stack Overflow platform from Kaggle. We did data pre-processing, data exploration and found out insights that could help us in our end goal to predict tags for each question. We performed several models and understood for which combination of the parameters we got the better predictions. Through this project, we were able to learn about Natural language processing and several other concepts such as Multi-label classification, Topic modelling etc.

11. CODES

```
#Loading the Necessary Modules
```

```
import pandas as pd
import numpy as np
```

```
pip install wordcloud
```

```
Requirement already satisfied: wordcloud in c:\users\ancha\anaconda3\lib\site-packages (1.8.1)
Requirement already satisfied: numpy>=1.6.1 in c:\users\ancha\anaconda3\lib\site-packages (from wordcloud) (1.20.1)
Requirement already satisfied: pillow in c:\users\ancha\anaconda3\lib\site-packages (from wordcloud) (8.2.0)
Requirement already satisfied: matplotlib in c:\users\ancha\anaconda3\lib\site-packages (from wordcloud) (3.3.4)
Requirement already satisfied: pyparsing!=2.0.4,!=2.1.2,!=2.1.6,>=2.0.3 in c:\users\ancha\anaconda3\lib\site-packages (from matplotlib->wordcloud) (2.4.7)
Requirement already satisfied: python-dateutil>=2.1 in c:\users\ancha\anaconda3\lib\site-packages (from matplotlib->wordcloud) (2.8.1)
Requirement already satisfied: cycler>=0.10 in c:\users\ancha\anaconda3\lib\site-packages (from matplotlib->wordcloud) (0.10.0)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\ancha\anaconda3\lib\site-packages (from matplotlib->wordcloud) (1.3.1)
Requirement already satisfied: six in c:\users\ancha\anaconda3\lib\site-packages (from cycler>=0.10->matplotlib->wordcloud) (1.15.0)
Note: you may need to restart the kernel to use updated packages.
```

```
import nltk
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\ancha\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

```
True
```

```
from nltk import word_tokenize
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
from collections import Counter
from wordcloud import WordCloud
```

```
import nltk
from nltk import FreqDist
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# importing the Necessary modules
from datetime import datetime
import matplotlib.pyplot as plt
import seaborn as sbn
import re
from sklearn.preprocessing import Normalizer
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem.snowball import SnowballStemmer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.multiclass import OneVsRestClassifier
from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import train_test_split
from scipy.sparse import hstack
from sklearn import metrics
import warnings
from wordcloud import WordCloud
```

```
warnings.filterwarnings('ignore')
stemmer=SnowballStemmer('english')
```

```
%%time
# number of duplicate values in the dataset is :
print("The Number of Duplicate values in the Dataset are :",train_data.duplicated().sum())
# Dropping the Duplicates in the dataset
train_data.drop_duplicates(inplace=True)
print("The dimension of the dataset after dropping the unwanted column & duplicate values :",train_data.shape)
```

```
#checking the null & removing the rows which contains the nul values in the dataset.
```

```
# Checking the Missing Values in the Dataset
print(train_data.isnull().sum())
# Here only contains single Null values so it better to remove that row instead repacing the null values.
train_data.dropna(inplace=True)
# The Shape of the Dataset after Removing the rows which Contains Null vales
train_data.shape
```

```
#Tags Analysis
```

```
#Counting the Tags for each Question or Row in the Dataset
train_data["Count_tags"]=train_data.Tags.apply(lambda x:len(str(x).split())) # counting for each Query
```

```
# Distribution of Tags
sbn.countplot(train_data.Count_tags)
plt.title("Number of Tags associated with the queries")
plt.xlabel("Number of Tags")
plt.ylabel("Number of Queries")
```

```
#Maximum Tags in query is : 5
#Most of the Questions Contains 3 or 2 tags.
```

```
#Analyzing how tags are Frequent in a corpus.
#Getting the most frequent tags in the corpus.
```

```
# To get the frequent tags by the Countvectorizer
# GETTING THE Most Frequent Tags in the corpus.
tags_vector=CountVectorizer(tokenizer = lambda x: x.split(),binary=True)
x_tags=tags_vector.fit_transform(train_data.Tags)
count=x_tags.sum(axis=0)
freq_tags=pd.DataFrame()
freq_tags["tags"]=tags_vector.get_feature_names()
freq_tags["count"]=count.tolist()[0]
freq_tags.sort_values(by='count',ascending=False,inplace=True)
freq_tags.reset_index(inplace=True)
freq_tags.drop(columns="index",inplace=True)
```

```
print("-----The Top 10 Tags in the Corpus -----")
print(freq_tags["tags"][:10].values)
print("-----Top 10 Least Frequent tags in the Corpus-----")
print(freq_tags.tail(10)["tags"].values)
```

```
#data preprocessing
# function to plot most frequent terms
def freq_words(x, terms = 10):
    all_words = ' '.join([text for text in x])
    all_words = all_words.split()
    fdist = FreqDist(all_words)
    words_df = pd.DataFrame({'word':list(fdist.keys()), 'count':list(fdist.values())})
    # selecting top 10 most frequent words
    d = words_df.nlargest(columns="count", n = terms)
    plt.figure(figsize=(20,5))
    ax = sns.barplot(data=d, x= "word", y = "count")
    ax.set(ylabel = 'count')
    plt.show()
```

```
#Sorting the tags based on counts.
freq_tags.sort_values(['count'], ascending=False, inplace=True)
freq_tags.head(20)
# 20 most frequent tags
```

```
# frequency plotting of the Tags
sbn.set_style("whitegrid")
sbn.lineplot(data=freq_tags,x=freq_tags.index,y="count")
plt.title("Tag Occurrence in Corpus",color="blue")
plt.xlabel("Tag numbers for most frequent tags")

#Large number of tags has occurred rarely in the Corpus.

#If we observe, the first few tag occurs more than 300000
#times and last tag occurs only one time. By looking at
#the diagram we can say that this is a highly skewed distribution.
```

```
#Plotting the Top 500 Frequent tags.
```

```
sbn.set_style("dark")
sbn.lineplot(data=freq_tags.iloc[:100],x=freq_tags.index[:100],y="count")
plt.title("Occurrence of Tags in corpus",color="green")
plt.xlabel("Tag numbers for most frequent tags")

#Here are Looking for the top 300 frequent tags , we can see that
#1st tag has occurred more than 7500+ times & the 100th tag has occurred
#between 0 to 1000 times .
```

```
# Lets plot the Top 100 Tags in the Corpus
plt.figure(figsize=(10,7))
sbn.set_style("darkgrid")
sbn.lineplot(data=freq_tags.iloc[:100],x=freq_tags.index[:100],y="count")
a=[0,20,40,60,80,100]
sbn.scatterplot(a,freq_tags.iloc[a,1],hue=freq_tags.iloc[a,0])
plt.title("Occurrence of Tags in corpus",color="green")
plt.xlabel("Tag numbers for most frequent tags")
plt.ylabel("Frequency")
plt.legend()
```

```
# creating the Dictionary with the frequent words in the dataset
t=freq_tags.tags.to_list()
c=freq_tags["count"].to_list()
tags_dic={}
for i,j in zip(t,c):
    tags_dic[i]=j

# Create and generate a word cloud image:
wordcloud = WordCloud().generate_from_frequencies(tags_dic)
plt.figure(figsize=(10,10))
# Display the generated image:
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.show()

#by the word cloud C#,java,Php,JavaScript
#android python are the occurring vary frequently in the corpus.
```

```
%%time
char_title_pre=train_data.Title.apply(lambda x:len(x))
char_body_pre=train_data.Body.apply(lambda x:len(x))
code_body_pre=train_data.Body.apply(lambda x:len(re.findall(r"<code>",x)))
words_title_pre=train_data.Title.apply(lambda x:len(str(x).split()))
```

```
# Defining Some Extra StopWords seems to be Not useful
extra=['could','would','lis','sometimes','sometime','puts','put','get','gets','help','please','need',\
       'like','know','thank','thanks','madam','sir','hii','doubt','doubts','www','com']
li=stopwords.words("english")
li=li+extra
```

```
# creating the Dictionary with the frequent words in the dataset
t=freq_tags.tags.to_list()
c=freq_tags["count"].to_list()
tags_dic={}
for i,j in zip(t,c):
    tags_dic[i]=j

# Create and generate a word cloud image:
wordcloud = WordCloud().generate_from_frequencies(tags_dic)
plt.figure(figsize=(10,10))
# Display the generated image:
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.show()

#by the word cloud C#,java,Php,JavaScript
#android python are the occurring vary frequently in the corpus.
```

```
#Preprocessing
# removing the Html , tags urls
#removing the code segments
#removing the punctuations & numbers & keeping only Text.

def Preprocessor(doc):
    body=[]
    for text in doc:
        text=re.sub(r"href.*", "",text) #removing the href ie. removing the hyper Links
        text=re.sub('<code>(.*?)</code>', '', text, flags=re.MULTILINE|re.DOTALL) # removing the code segments
        text=re.sub('<.*?>', '', str(text.encode('utf-8')))) #removing the Html Tags in the Text
        text=re.sub(r"[a-zA-Z]+", "",text) # removing numbers and most of Puncutations in the Text.
        text=text.lower() # converting from upper case to lower case
        body.append(" ".join([k for k in text.split() if((len(k)>2 or k!="c") and k not in li)]))

    return body
```

```
%%time
pre_body=Preprocessor(train_data.Body) ## Preprocessing the Body Columns
pre_text=Preprocessor(train_data.Title) ## Preprocessing the Title Columns
```

```
## Replacing the a Title and Body with the Preprocessed Title and Body respectively.
train_data["Title"]=pre_text
train_data["Body"]=pre_body
pre_data=train_data # Creating a Reference for the train_data_100k
```

```
#defining Meta Features after preprocessing the dataset.

words_title_post=train_data.Title.apply(lambda x:len(str(x).split()))
words_body_post=train_data.Body.apply(lambda x:len(str(x).split()))
```

```
## seperate the tag columns and the dropping the Tags column from the dataset
y_tagss=train_data.Tags
train_data.drop(columns="Tags",axis=1,inplace=True)
```

```
## reference to the pre_data
pre_data=train_data
```

```
%%time
# Converting the Tags columns in to the Multilabel Classification
# initializing the Count vectorizer
tag_vect=CountVectorizer(binary=True,tokenizer=lambda x:str(x).split(),max_features=500)
vec_tag=tag_vect.fit_transform(y_tagss)

Wall time: 16.5 s
```

```
# Converting string Tags to multilable output variables
# binary='true' will give a binary vectorizer
vectorizer = CountVectorizer(tokenizer = lambda x: x.split(), binary='true')
multilabel_y = vectorizer.fit_transform(y_tagss)
```

```

def tags_to_choose(n):
    t = multilabel_y.sum(axis=0).tolist()[0]
    sorted_tags_i = sorted(range(len(t)), key=lambda i: t[i], reverse=True)
    multilabel_yn = multilabel_y[:,sorted_tags_i[:n]]
    return multilabel_yn

def questions_explained_fn(n):
    multilabel_yn = tags_to_choose(n)
    x = multilabel_yn.sum(axis=1)
    return (np.count_nonzero(x==0))

questions_explained = []
total_tags = multilabel_y.shape[1]
total_qs = train_data.shape[0]
for i in range(500, total_tags, 100):
    questions_explained.append(np.round(((total_qs-questions_explained_fn(i))/total_qs)*100,3))

fig, ax = plt.subplots()
ax.plot(questions_explained)
xlabel = list(500 + np.array(range(-50,450,50))*50)
ax.set_xticklabels(xlabel)
plt.xlabel("Number of tags")
plt.ylabel("Number Questions covered partially")
plt.grid()
plt.show()

# You can choose any number of tags based on your computing power, minimum is 50(it covers 90% of the tags)
print("With ", 5500, "tags we are covering ", questions_explained[50], "% of questions")

multilabel_yx = tags_to_choose(500)
print("Number of questions that are not covered:", questions_explained_fn(500),"out of", total_qs)

Number of questions that are not covered: 258537 out of 2525238

# We consider top 15% tags which covers 99% of the questions
print("Number of tags in sample:", multilabel_y.shape[1])
print("Number of tags taken:", multilabel_yx.shape[1],"(",(multilabel_yx.shape[1]/multilabel_y.shape[1])*100,"%")")

Number of tags in sample: 39975
Number of tags taken: 500 ( 1.2507817385866167 %)

total_size = train_data.shape[0]
train_size = int(0.80*total_size)

x_train = train_data.head(train_size)
x_test = train_data.tail(total_size - train_size)

y_train = multilabel_yx[0:train_size, :]
y_test = multilabel_yx[train_size:total_size, :]

print("Number of data points in train data:", y_train.shape)
print("Number of data points in test data:", y_test.shape)

start = datetime.now()
vectorizer = TfidfVectorizer(min_df=0.00009, max_features=200000, smooth_idf=True, norm="l2", \
                             tokenizer = lambda x: x.split(), sublinear_tf=False, ngram_range=(1,3))
x_train_multilabel = vectorizer.fit_transform(x_train['title'])
x_test_multilabel = vectorizer.transform(x_test['title'])
print("Time taken to run this cell :", datetime.now() - start)

Time taken to run this cell : 0:02:30.296796

print("Dimensions of train data X:",x_train_multilabel.shape, "Y :",y_train.shape)
print("Dimensions of test data X:",x_test_multilabel.shape,"Y:",y_test.shape)

Dimensions of train data X: (2020190, 7870) Y : (2020190, 500)
Dimensions of test data X: (505048, 7870) Y: (505048, 500)

# print percent of question covered with number of tags
print('#Tags\t%Ques')
for idx in range(500, 7500, 500):
    print(idx, '\t', questions_explained[int(idx/100)])

```

MODELS

OneVsRestClassifier (SGDClassifier)LABELS=500

```

x_train_multilabel
<2020190x7870 sparse matrix of type '<class 'numpy.float64''>
  with 10870259 stored elements in Compressed Sparse Row format>

classifier = OneVsRestClassifier(SGDClassifier(loss='log', alpha=0.00001, penalty='l1', n_jobs=-1))
classifier.fit(x_train_multilabel, y_train)
predictions = classifier.predict(x_test_multilabel)

print("accuracy :",metrics.accuracy_score(y_test,predictions))
print("macro f1 score :",metrics.f1_score(y_test, predictions, average = 'macro'))
print("micro f1 score :",metrics.f1_score(y_test, predictions, average = 'micro'))
print("hamming loss :",metrics.hamming_loss(y_test,predictions))
print("Precision recall report :\n",metrics.classification_report(y_test, predictions))

```

OneVsRestClassifier (LogisticRegression)LABELS=500

```

classifier_2 = OneVsRestClassifier(LogisticRegression(loss='log', alpha=0.00001, penalty='l1'), n_jobs=-1)
classifier_2.fit(x_train_multilabel, y_train)
predictions_2 = classifier_2.predict(x_test_multilabel)

print("accuracy :",metrics.accuracy_score(y_test,predictions_2))
print("macro f1 score :",metrics.f1_score(y_test, predictions_2, average = 'macro'))
print("micro f1 score :",metrics.f1_score(y_test, predictions_2, average = 'micro'))
print("hamming loss :",metrics.hamming_loss(y_test,predictions_2))
print("Precision recall report :\n",metrics.classification_report(y_test, predictions_2))

```

OneVsRestClassifier (SGDClassifier)LABELS=500 title 3X

```

#train_data.to_csv(r'C:/Users/ancha/OneDrive/Desktop/cleandata.csv')
data=pd.read_csv('C:/Users/ancha/OneDrive/Desktop/6th Sem Project/clean.csv')

data["Question"]=data["Title"].str.cat(data["Title"],sep=" ")

data["Question"]=data["Title"].str.cat(data["Title"],sep=" ")

data["Question"]=data["Question"].str.cat(data["Title"],sep=" ")

#data["question"].head()
data["Question"].head()

train_data=data
train_data.head()

x_train_multilabel

<2020190x7870 sparse matrix of type '<class 'numpy.float64'>'
with 10870259 stored elements in Compressed Sparse Row format>

classifier = OneVsRestClassifier(SGDClassifier(loss='log', alpha=0.00001, penalty='l1'), n_jobs=-1)
classifier.fit(x_train_multilabel, y_train)
predictions = classifier.predict(x_test_multilabel)

print("accuracy :",metrics.accuracy_score(y_test,predictions))
print("macro f1 score :",metrics.f1_score(y_test, predictions, average = 'macro'))
print("micro f1 score :",metrics.f1_score(y_test, predictions, average = 'micro'))
print("hamming loss :",metrics.hamming_loss(y_test,predictions))
print("Precision recall report :\n",metrics.classification_report(y_test, predictions))

```

OneVsRestClassifier (LogisticRegression)LABELS=500 title 3X

```

classifier_2 = OneVsRestClassifier(LogisticRegression(loss='log', alpha=0.00001, penalty='l1'), n_jobs=-1)
classifier_2.fit(x_train_multilabel, y_train)
predictions_2 = classifier_2.predict(x_test_multilabel)

print("accuracy :",metrics.accuracy_score(y_test,predictions_2))
print("macro f1 score :",metrics.f1_score(y_test, predictions_2, average = 'macro'))
print("micro f1 score :",metrics.f1_score(y_test, predictions_2, average = 'micro'))
print("hamming loss :",metrics.hamming_loss(y_test,predictions_2))
print("Precision recall report :\n",metrics.classification_report(y_test, predictions_2))

```

OneVsRestClassifier (SGDClassifier)LABELS=5000

```

classifier = OneVsRestClassifier(SGDClassifier(loss='log', alpha=0.00001, penalty='l1'), n_jobs=-1)
classifier.fit(x_train_multilabel, y_train)
predictions = classifier.predict(x_test_multilabel)

print("accuracy :",metrics.accuracy_score(y_test,predictions))
print("macro f1 score :",metrics.f1_score(y_test, predictions, average = 'macro'))
print("micro f1 score :",metrics.f1_score(y_test, predictions, average = 'micro'))
print("hamming loss :",metrics.hamming_loss(y_test,predictions))
print("Precision recall report :\n",metrics.classification_report(y_test, predictions))

```

oneVsRestClassifier (LogisticRegression)LABELS=5000

```

classifier_2 = OneVsRestClassifier(LogisticRegression(loss='log', alpha=0.00001, penalty='l1'), n_jobs=-1)
classifier_2.fit(x_train_multilabel, y_train)
predictions_2 = classifier_2.predict(x_test_multilabel)

print("accuracy :",metrics.accuracy_score(y_test,predictions_2))
print("macro f1 score :",metrics.f1_score(y_test, predictions_2, average = 'macro'))
print("micro f1 score :",metrics.f1_score(y_test, predictions_2, average = 'micro'))
print("hamming loss :",metrics.hamming_loss(y_test,predictions_2))
print("Precision recall report :\n",metrics.classification_report(y_test, predictions_2))

```


12. BIBLIOGRAPHY

- https://en.wikipedia.org/wiki/Stack_Overflow
- <https://medium.datadriveninvestor.com/predicting-tags-for-the-questions-in-stack-overflow-29438367261e>
- <https://www.kaggle.com/competitions/facebook-recruiting-iii-keyword-extraction/data>
- <https://www.analyticsvidhya.com/blog/2020/12/hands-on-tutorial-on-stack-overflow-question-tagging/>
- <https://arxiv.org/pdf/1911.12224.pdf>
- https://www.sas.com/en_us/insights/analytics/what-is-natural-language-processing-nlp.html
- <https://www.analyticsvidhya.com/blog/2021/09/onevsrest-classifier-for-predicting-multiple-tags-of-research-articles/>
-